PRUEBAS DE ANALISIS ESTÁTICO

CMD: pip install ruff

NEW FILE: analisis.py

```python
import math

def calcular_total(precio, descuento):
    resultado = precio - precio * descuento / 100
    temporal = 500
    return resultado

def obtener_categoria(precio):
    if precio > 1000:
        return "Premium"
    else:
        return "Normal"
```

CMD: python -m ruff check analisis.py

CMD: python -m ruff check analisis.py –fix

PRUEBAS DE ACEPTACIÓN

HISTORIA DE USUARIO

Como cliente de la tienda, quiero conocer el costo del envío antes de confirmar mi compra, para saber cuánto pagaré en total.

Criterios de aceptación:

- Pedidos con subtotal de $500 o más: envío gratuito.
- Pedidos con subtotal inferior a $500: envío de $80.
- No se aceptan subtotales negativos.
- El subtotal se calcula después del descuento y antes del IVA.

On **tienda.py**:

```
def calcular_envio(subtotal):
    if subtotal < 0:
        raise ValueError("Subtotal invalido")
    if subtotal > 500:
        return 0
    return 80
```

NEW FILE: test_aceptacion.py

```
import pytest
from tienda import calcular_envio

@pytest.mark.parametrize(
    "subtotal,envio_esperado",
    [
        (300, 80),
        (499, 80),
        (500, 80),
        (600, 0),
    ]
)
def test_reglas_de_envio(subtotal, envio_esperado):
    assert calcular_envio(subtotal) == envio_esperado
def test_subtotal_negativo():
    with pytest.raises(ValueError):
        calcular_envio(-100)
```

CMD: python -m pytest test_aceptacion.py -v

PRUEBAS DE REGRESIÓN

On **tienda.py** – función original

```
def calcular_precio(precio, descuento):
    validar_descuento(descuento)
    total = precio - (precio * descuento / 100)
    return total
```

NEW FILE: **test_regresión.py**

```
import pytest
from tienda import calcular_precio

@pytest.mark.parametrize(
    "precio,descuento,esperado",
    [
        (200, 25, 150),
        (500, 0, 500),
        (100, 100, 0),
        (300, 10, 270),
    ]
)
def test_descuentos_existentes(
    precio, descuento, esperado
):
    assert calcular_precio(
        precio, descuento
    ) == pytest.approx(esperado)
```

CMD: **python -m pytest test_regresion.py -v**

(pasan todas)

On **tienda.py** – modificar función

```
def calcular_precio(precio, descuento):
    validar_descuento(descuento)
    if descuento == 0:
        return precio
    if descuento == 100:
        return 0
    return precio - precio * descuento / 10
```

PRUEBAS DE ESFUERZO (STRESS)

Instalar fastapi, uvicorn y locust

- FastAPI: permite construir nuestra API.
- Uvicorn: ejecuta el servidor de la aplicación.
- Locust: genera solicitudes simulando usuarios concurrentes.

CMD: python -m pip install fastapi uvicorn locust

NEW FILE: api.py

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI()

class Compra(BaseModel):
    precio: float
    descuento: float

@app.post("/compra")
def procesar_compra(compra: Compra):
    if compra.precio < 0:
        raise HTTPException(400, "Precio invalido")

    if not 0 <= compra.descuento <= 100:
        raise HTTPException(400, "Descuento invalido")

    subtotal = compra.precio * (
        1 - compra.descuento / 100
    )

    total = round(subtotal * 1.16, 2)
    return {"total": total}
```

CMD: python -m uvicorn api:app --host 127.0.0.1 --port 8000

Browser: http://127.0.0.1:8000/docs

NEW FILE: **locustfile.py**

```python
from locust import HttpUser, task, between

class ClienteTienda(HttpUser):
    wait_time = between(1, 2)

    @task
    def realizar_compra(self):
        self.client.post(
            "/compra",
            json={
                "precio": 200,
                "descuento": 25
            }
        )
```

CMD: **python -m locust --host http://127.0.0.1:8000**

http://localhost:8089

Paso 5. Interpretar los resultados

Locust mostrará métricas como las siguientes:

Métrica	**Qué significa**
Requests/s	**Solicitudes procesadas por segundo.**
Average response time	**Tiempo promedio de respuesta.**
p95	**Tiempo bajo el cual se completó aproximadamente el 95 % de las solicitudes.**
Failures	**Solicitudes registradas como fallidas.**
Concurrent users	**Usuarios virtuales activos en la simulación.**

RPS – requests per second